

Fundamentals of Data Science

Master's Degree in Data Science, Sapienza Università di Roma
Faculty of Information Engineering, Informatics and Statistics

Department of Computer Science

Author: Gianmaria Romano

Reference Bibliography

M. P. Deisenroth, A. A. Faisal, C. S. Ong: *Mathematics for Machine Learning*
S. Prince: *Understanding Deep Learning*
J. Watt, R. Borhani, A. K. Katsaggelos: *Machine Learning Refined*
W. McKinney: *Python for Data Analysis*
M. E. J. Newman: *Networks: An Introduction*

Academic Year 2026/2027

Disclaimer

This document contains notes for the **Fundamentals of Data Science** course, given by **Professors Cinelli and Spinelli** as part of the Master's Degree in Data Science at **Sapienza Università di Roma** during the **first** semester of Academic Year **2026/2027**.

Note: These notes may not cover all details of each lecture, so it is highly suggested to consult the Professor's official resources and materials.

Important: These notes are free to share and use, but please remember to credit me as the author and do not try to hide/remove this page.

Contacts

For further information or requests, feel free to contact me privately or reach out through my repository: <https://github.com/GianmariaRomano/Data-Science-Notes>.

License

These documents are distributed under the GNU General Public License 3.0, a form of copyleft intended for use on a manual, textbook or other documents. Material licensed under the current version of the license can be used for any purpose, as long as the use meets certain conditions:

- All previous authors of the work must be attributed.
- All changes to the work must be logged.
- All derivative works must be licensed under the same license.
- The full text of the license, unmodified invariant sections as defined by the author if any, and any other added warranty disclaimers (such as a general disclaimer alerting readers that the document may not be accurate for example) and copyright notices from previous versions must be maintained.
- Technical measures such as DRM may not be used to control or obstruct distribution or editing of the document.

Contents

1	Introduction to data science	4
1.1	The lifecycle of data science	4
1.2	The role of machine learning in data science	4
1.3	Supervised learning	5
1.4	Unsupervised learning	5
1.5	Reinforcement learning	5
2	Vectors	7
2.1	Representing data using vectors	7
2.2	Spanning sets and bases	7
2.2.1	Standard bases	8
2.2.2	Non-standard bases	8
2.3	Vector subspaces	9
3	Optimization methods	11
3.1	The goal of optimization	11
3.1.1	Zero-order optimization	11
3.1.2	First-order optimization	11
3.2	Hyperplanes	12
I	Machine Learning	13
4	Principal component analysis	14
4.1	Linear transformations	14
4.2	Learning the projection basis	14
4.3	Optimizing a single basis vector	15
4.3.1	Eigenvalue decompositions	17
4.4	Applications of principal component analysis	17
II	Network Science	18
5	Understanding networks	19
5.1	The purpose of network science	19

5.2	A formal definition of networks	20
5.2.1	Visualizing a network using an edge list	20
5.2.2	Visualizing a network using an adjacency matrix	21
5.2.3	Visualizing a network using an adjacency list	21

Chapter 1

Introduction to data science

1.1 The lifecycle of data science

Data science can be seen as an interdisciplinary field that focuses on extracting actionable insights and knowledge from structured and unstructured data using techniques based on statistical and mathematical methods, as well as computer science algorithms and domain-specific knowledge.

Generally speaking, a data science pipeline starts with understanding the business objectives and translating them into machine learning objectives.

Next, after collecting and cleaning data, the pipeline focuses on selecting and training a model and evaluating the results for deployment, ultimately monitoring model efficacy and restarting the pipeline, either by collecting new data or re-training the model, whenever needed.

1.2 The role of machine learning in data science

Artificial intelligence is a sub-field of computer science that focuses on creating agents that can perform tasks requiring human intelligence and problem solving.

More specifically, Tom Mitchell defines machine learning as “the study of algorithms that improve their performance P at some task T with experience E ”, typically focusing on tasks that would be too complex to just hard-code.

Example: Playing checkers

The task $\langle P, T, E \rangle$ to train an agent to play checkers can be defined using the following components:

- The agent’s performance P is evaluated as the proportion of matches it wins against an arbitrary opponent.
- The task T to perform is to train the model to make the correct moves during a match.
- The experience E used to train the agent comes from matches the agent plays against itself.

While machine learning and statistics typically use the same instruments and algorithms to discover patterns within data, the two fields are normally considered to be different areas because statistics follows a more rigorous inference-based approach, while machine learning focuses on making predictions using scalable and autonomous techniques.

1.3 Supervised learning

In a supervised learning task, the model is given some input examples $(\mathbf{x}, y)$ with the aim of learning a function $\hat{y} = h(\mathbf{x})$ that will be used to predict the target variable based on new, unseen data. In particular, a supervised learning task can be formulated as a classification problem, which focuses on prediction discrete classes based on a probability distribution, or as a regression problem, which aims to predict continuous values.

Example: Some supervised learning tasks

Common classification tasks can include binary classification on the sentiment of a review or multi-class classification of which cat breed is featured in an image.

On the other hand, regression tasks can include univariate prediction of house prices or bivariate prediction of the freezing and boiling points of a chemical.

1.4 Unsupervised learning

In an unsupervised task, a model uses some input data $\mathbf{x}$ to discover patterns and structural relationships or to generate new data samples.

Example: Generative models as unsupervised learning

Generative models, such as generative adversarial networks or (variational) autoencoders, are considered to be an example of unsupervised learning because they learn how to create new data by sampling information from a distribution of unlabeled data.

1.5 Reinforcement learning

A reinforcement task is defined using a sequence of states, actions and rewards that the agent uses to find a policy that allows it to maximize its overall reward.

In this setting, the main challenges come from the stochastic nature of the task, which means that the environment might react differently to the same action, and the trade-off between exploitation and exploration as the agent needs to determine whether to stick to a known good policy or try out new actions in the hopes of obtaining a better reward.

Example: Chess as a reinforcement learning task

It is possible to train an agent to play chess using reinforcement learning. In fact, the idea is to consider the task as a configuration (S, A, R) in which S represents the set of possible valid states and A represents the set of possible legal moves, while the reward R will be positive whenever the agent manages to take the opponent's pieces and negative whenever the agent loses its pieces.

Chapter 2

Vectors

2.1 Representing data using vectors

When dealing with supervised learning tasks, a dataset consists of measurements of some features of objects, which are used to predict a target variable.

However, while features can be represented as numerical, categorical or binary variables, most machine learning algorithms require numerical inputs.

For this reason, categorical variables are often transformed into numerical representations through one-hot encoding, which, in the case of mutually exclusive classes, represents each feature using a vector of type $\mathbf{e}_i = (0, \dots, 1, \dots, 0)^T$, where the position of the single non-zero entry identifies the class to which the observation belongs.

Example: The IRIS Dataset

Fisher’s IRIS Dataset is a toy example of a dataset that classifies each flower into one of three possible subspecies based on petal and sepal measurements.

From a mathematical perspective, a dataset can be described as a set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n \subset (X \times Y)^n$, where $X \subseteq \mathbb{R}^p$ is known as the “input space” while Y is known as the target space, or as a matrix whose rows represent single observations and whose columns denote specific features.

In particular, it is assumed that the dataset is generated from an unknown distribution $P_{X,Y}$ on $X \times Y$, which the model aims to (partially) learn, and that each observed sample $(\mathbf{x}^{(i)}, y^{(i)})$ is drawn independent and identically distributed from $P_{X,Y}$, which means that all samples are mutually independent and come from the same probability distribution.

2.2 Spanning sets and bases

Assuming a dataset has been mean-centered, each observed sample can be seen as a data point in the corresponding input vector space.

From a broader perspective, a collection of vectors is said to form a spanning set if combining the vectors using addition and scalar multiplication allows to create every vector in the corresponding vector space.

In particular, a basis is any minimal spanning set that, having no redundant vectors, satisfies the condition of linear independence.

Example: Spanning sets in $\mathbb{R}^2$

In the vector space $\mathbb{R}^2$, a possible spanning set can be composed using the canonical vectors $\mathbf{e}_1 = (1, 0)^T$ and $\mathbf{e}_2 = (0, 1)^T$ as each point $(x, y) \in \mathbb{R}^2$ can be reformulated as $x\mathbf{e}_1 + y\mathbf{e}_2$. Additionally, it is possible to observe that this spanning set also works as a basis for $\mathbb{R}^2$ because its vectors are linearly independent.

Generally speaking, a candidate basis $\mathbf{c}_1, \dots, \mathbf{c}_N \in \mathbb{R}^N$ is able to perfectly represent a dataset $\{\mathbf{x}_i\}_{i=1}^P \in \mathbb{R}^N$ if it is possible to find some weights such that

$$\sum_{n=1}^N \mathbf{c}_n w_{n,p} = \mathbf{x}_p \quad \forall p \in \{1, \dots, P\}$$

2.2.1 Standard bases

The canonical vectors $\mathbf{e}_1, \dots, \mathbf{e}_N \in \mathbb{R}^N$ always form a standard basis such that

$$\mathbf{e}_n^T \mathbf{e}_m = 0 \quad \forall n \neq m \quad \text{and} \quad \|\mathbf{e}_n\|_2^2 = 1 \quad \forall n \in \{1, \dots, N\}$$

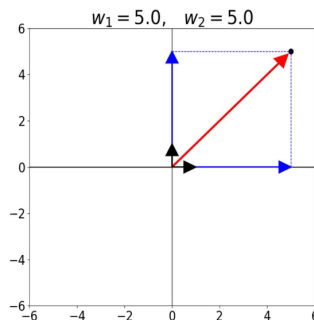


Figure 2.1: Example of a standard basis in $\mathbb{R}^2$.

In this case, representing a vector $\mathbf{x}_p$ is a trivial task as the encoding weights will always be given by $\mathbf{w}_p = \mathbf{x}_p$.

It should be noticed that all these properties can also be expressed using the concatenated basis matrix $\mathbf{C}$: knowing that $\mathbf{C}^T \mathbf{C} = \mathbf{C} \mathbf{C}^T = \mathbf{I}_{N \times N}$, it is in fact possible to find that

$$\mathbf{w}_p = \mathbf{C}^T \mathbf{x}_p \quad \text{and} \quad \mathbf{C} \mathbf{C}^T \mathbf{x}_p = \mathbf{x}_p \quad \forall p \in \{1, \dots, P\}$$

2.2.2 Non-standard bases

When dealing with a non-standard basis, the new vector $\mathbf{w}_p$ acts as the new encoding of $\mathbf{x}_p$ in the transformed feature space whose coordinate axes are defined by the spanning set.

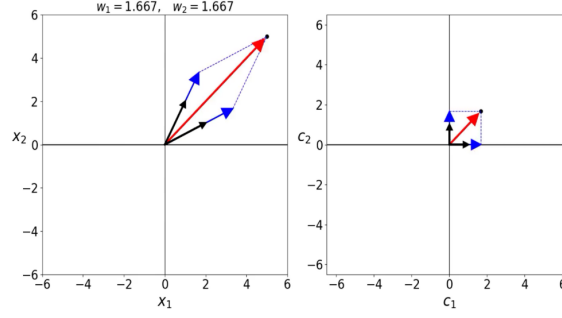


Figure 2.2: Example of a non-standard basis in $\mathbb{R}^2$.

Therefore, using the concatenated basis matrix, the goal is to find the encoding that solves the linear system of equations

$$\mathbf{C}^T \mathbf{C} \mathbf{w}_p = \mathbf{C}^T \mathbf{x}_p \quad \forall p \in \{1, \dots, P\}, \text{ which is equivalent to solving}$$

$$\min_{\mathbf{w}_1, \dots, \mathbf{w}_P} \frac{1}{P} \sum_{p=1}^P \|\mathbf{C} \mathbf{w}_p - \mathbf{x}_p\|_2^2, \text{ which represents the average reconstruction error.}$$

Example: A non-standard basis in $\mathbb{R}^2$

Consider a non-standard basis in $\mathbb{R}^2$ given by

$$\mathbf{c}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \text{ and } \mathbf{c}_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Expressing a vector using the equation $\mathbf{x}_p = w_{1,p} \mathbf{c}_1 + w_{2,p} \mathbf{c}_2$ reveals that this basis corresponds to a transformation that consists of a rotation followed by a vector shearing.

2.3 Vector subspaces

When representing N -dimensional data points $\{\mathbf{x}_p\}_p^P \in \mathbb{R}^N$ using $K \leq N$ basis vectors, the representation incurs in a loss of information because the data points are projected onto a lower-dimensional subspace.

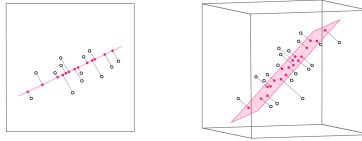


Figure 2.3: Projecting data points from $\mathbb{R}^3$ onto 1D and 2D subspaces.

In order to minimize the reconstruction error, each data point is projected orthogonally onto the subspace spanned by the basis vectors, meaning that the encoding weights will be computed by solving the linear system of equations

$$\mathbf{C}^T \mathbf{C} \mathbf{w}_p = \mathbf{C}^T \mathbf{x}_p, \text{ where } \mathbf{C} \in \mathbb{R}^{N \times K}.$$

In fact, this strategy ensures that, during decoding, the projected point provides the best possible approximation of the starting data point within the subspace.

Example: Projecting a vector from $\mathbb{R}^2$ onto a line

Suppose you want to project the vector $\mathbf{x} = (3, 1)^T$ onto the one-dimensional subspace spanned by the basis vector $\mathbf{c} = (1, 1)^T$.

Since the basis vector is not a unit vector, the encoding weight must be computed as

$$\begin{aligned} \mathbf{c}^T \mathbf{C} \mathbf{w} = \mathbf{c}^T \mathbf{x} &\Rightarrow \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} w = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix} \\ &\Rightarrow 2w = 4 \Rightarrow w = 2 \end{aligned}$$

Therefore, the reconstructed projection will be given by $w\mathbf{c} = (2, 2)^T$, resulting in a residual reconstruction error of

$$r = \|\mathbf{x} - w\mathbf{c}\|_2 = \left\| \begin{pmatrix} 3 \\ 1 \end{pmatrix} - \begin{pmatrix} 2 \\ 2 \end{pmatrix} \right\|_2 = \left\| \begin{pmatrix} 1 \\ -1 \end{pmatrix} \right\|_2 = \sqrt{2}$$

Whenever the chosen basis is orthonormal, the weight encodings can be computed using

$$\mathbf{w}_p = \mathbf{C}^T \mathbf{x}_p \text{ and } \mathbf{C} \mathbf{C}^T \mathbf{x}_p \approx \mathbf{x}_p \quad \forall p \in \{1, \dots, P\}$$

Chapter 3

Optimization methods

3.1 The goal of optimization

Given an objective function $g(\mathbf{w})$, optimization deals with finding the value $\mathbf{w}^*$ that minimizes or maximizes the function, which means that

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} g(\mathbf{w}) \text{ or } \mathbf{w}^* = \arg \max_{\mathbf{w}} g(\mathbf{w})$$

Additionally, maximizing an objective function $g(\mathbf{w})$ is always equivalent to minimizing $-g(\mathbf{w})$.

3.1.1 Zero-order optimization

Since zero-order optimization simply involves analysing the values taken by the objective function, it is possible to distinguish two types of solution:

- **Global optimum:** A point $\mathbf{w}^*$ is a global optimum if it represents a global maximum/minimum among every possible value of $\mathbf{w}$.
- **Local optimum:** A point $\mathbf{w}^*$ is a local optimum if it acts as a local maximum/minimum within some neighbourhood, but not necessarily among every possible value of $\mathbf{w}$.

Example: Zero-order optimization on a sinusoidal function

Consider the scalar function $g(w) = \sin(w)$: since this function has period equal to 2π , it is possible to observe that it has infinitely many global maxima and global minima.

More specifically the points $w^* = \frac{3\pi}{2} + 2k\pi$, with $k \in \mathbb{Z}$, act as global minima for the function, whereas the points $w^* = \frac{\pi}{2} + 2k\pi$, with $k \in \mathbb{Z}$, act as global maxima for the function.

3.1.2 First-order optimization

First-order optimization involves using (partial) derivatives to find candidate optima by exploiting the fact that, at an extremum of a differentiable function, it is always the case that

$$g'(w^*) = 0 \text{ or, for a multivariate function, } \nabla g(\mathbf{w}^*) = \mathbf{0}_{n \times 1}$$

Example: First-order optimization on a sinusoidal function

Consider the scalar function $g(w) = \sin(w)$: solving $g'(w) = \cos(w) = 0$ reveals that the points $w^* = \frac{\pi}{2} + k\pi$, with $k \in \mathbb{Z}$, act as extrema for the function.

Using the second derivative $g''(w) = -\sin(w)$ reveals that the points $w^* = \frac{3\pi}{2} + 2k\pi$ act as global minima for the function, whereas the points $w^* = \frac{\pi}{2} + 2k\pi$ act as global maxima for the function.

3.2 Hyperplanes

A hyperplane can be seen as a N -dimensional generalization of a line that can be defined as

$$h(\mathbf{w}) = a + \mathbf{b}^T \mathbf{w}$$

Since any point $\mathbf{w}_0$ admits infinitely many direction vectors $\mathbf{d}$, it is possible to find the direction of maximum increase or decrease by evaluating

$$h(\mathbf{w}_0 + \mathbf{d}) = a + \mathbf{b}^T(\mathbf{w}_0 + \mathbf{d}) = a + \mathbf{b}^T \mathbf{w}_0 + \mathbf{b}^T \mathbf{d}, \text{ which means that}$$

$$\max_{\mathbf{d}} h(\mathbf{w}_0 + \mathbf{d}) \text{ subject to } \|\mathbf{d}\|_2^2 = 1 \equiv \max_{\mathbf{d}} \mathbf{b}^T \mathbf{d} = \max_{\mathbf{d}} \|\mathbf{b}\|_2^2 \|\mathbf{d}\|_2^2 \cos(\theta) \stackrel{\|\mathbf{d}\|=1}{=} \max_{\theta} \cos(\theta)$$

Solving this problem reveals that the maximum increase happens whenever $\theta = 0$, which corresponds to the direction vector $\mathbf{d} = \frac{\mathbf{b}}{\|\mathbf{b}\|_2^2}$, whereas the maximum decrease happens whenever $\theta = \pi$, which corresponds to the direction vector $\mathbf{d} = -\frac{\mathbf{b}}{\|\mathbf{b}\|_2^2}$.

Part I

Machine Learning

Chapter 4

Principal component analysis

4.1 Linear transformations

Given a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, a linear transformation $\mathbf{y} = \mathbf{Ax}$ maps a vector $\mathbf{x} \in \mathbb{R}^n$ to a vector $\mathbf{y} \in \mathbb{R}^m$.

- **Scaling:** If $\mathbf{A}$ is a diagonal matrix, then the transformation $\mathbf{y} = \mathbf{Ax}$ simply stretches or shrinks each axis by the corresponding factor.
- **Rotation:** If $\mathbf{A}$ is an orthonormal matrix, meaning that $\mathbf{A}^T \mathbf{A} = \mathbf{I}$, then the transformation $\mathbf{y} = \mathbf{Ax}$ represents a rotation or a reflection.
- **Shearing:** If $\mathbf{A}$ features some off-diagonal entries, then the transformation $\mathbf{y} = \mathbf{Ax}$ typically results in a slanted vector.
- **Projection:** If $\mathbf{A}$ has rank $k < n$, then the transformation $\mathbf{y} = \mathbf{Ax}$ always results in a loss of information because the vectors collapse into a lower-dimensional space.

4.2 Learning the projection basis

Rather than finding the encoding weights for a given basis, principal component analysis aims to learn both the encoding weights $\mathbf{w}_1, \dots, \mathbf{w}_p$ and the basis matrix $\mathbf{C}$ that minimize the mean reconstruction error

$$g(\mathbf{w}_1, \dots, \mathbf{w}_p, \mathbf{C}) = \frac{1}{P} \sum_{p=1}^P \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2, \text{ for a dataset } \{\mathbf{x}_p\}_{p=1}^P.$$

However, if $\mathbf{C}$ is assumed to be orthonormal, it is possible to directly compute $\mathbf{w}_p = \mathbf{C}^T \mathbf{x}_p$ and simplify the objective function as

$$g(\mathbf{C}) = \frac{1}{P} \sum_{p=1}^P \|\mathbf{C}\mathbf{C}^T \mathbf{x}_p - \mathbf{x}_p\|_2^2$$

Proof:

Assuming $\mathbf{C}$ is orthonormal, meaning that $\mathbf{C}^T \mathbf{C} = \mathbf{I}$, the cost function can be formulated as

$$\begin{aligned} \|\mathbf{C}\mathbf{C}^T \mathbf{x}_p - \mathbf{x}_p\|_2^2 &= \mathbf{x}_p^T \mathbf{C}^T \mathbf{C} \mathbf{C} \mathbf{C}^T \mathbf{x}_p - 2\mathbf{x}_p^T \mathbf{C} \mathbf{C}^T \mathbf{x}_p + \mathbf{x}_p^T \mathbf{x}_p \\ &\stackrel{\mathbf{C}^T \mathbf{C} = \mathbf{I}}{=} -\mathbf{x}_p^T \mathbf{C} \mathbf{C}^T \mathbf{x}_p + \mathbf{x}_p^T \mathbf{x}_p = \|\mathbf{C}^T \mathbf{x}_p\|_2^2 + \|\mathbf{x}_p\|_2^2 \end{aligned}$$

Since $\mathbf{x}_p$ is fixed with respect to the optimization over the basis matrix, minimizing the mean reconstruction error is equivalent to maximizing $\|\mathbf{C}^T \mathbf{x}_p\|$.

In particular, the assumption that the basis is orthonormal allows to learn each vector independently as the objective function decouples into

$$g(\mathbf{C}) = -\frac{1}{P} \sum_{p=1}^P \sum_{k=1}^K (\mathbf{c}_k^T \mathbf{x}_p)^2$$

4.3 Optimizing a single basis vector

Assuming the dataset has been mean-centered and that the basis vector $\mathbf{c}_1$ has unit norm, the quantity

$$h(\mathbf{c}_1) = \frac{1}{P} \sum_{p=1}^P (\mathbf{c}_1^T \mathbf{x}_p)^2 \text{ represents the sample variance along direction } \mathbf{c}_1.$$

By stacking the data points into a matrix $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_P] \in \mathbb{R}^{N \times P}$, one can reformulate this quantity as

$$h(\mathbf{c}_1) = \frac{1}{P} \mathbf{c}_1^T \mathbf{X} \mathbf{X}^T \mathbf{c}_1 = \mathbf{c}_1^T \left(\frac{1}{P} \mathbf{X} \mathbf{X}^T \right) \mathbf{c}_1 = \mathbf{c}_1^T \mathbf{\Sigma} \mathbf{c}_1$$

In this setting, $\mathbf{\Sigma}$ is known as the “empirical covariance matrix” and, by describing how features vary together, it provides information about the shape of the data cloud through the eigenvectors, which indicate the principal directions in which data spreads the most, and the eigenvalues, which explain the variance along these directions.

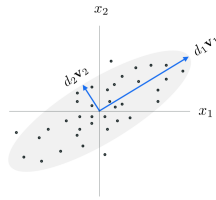


Figure 4.1: Principal component analysis as a dataset rotation.

Therefore, $h(\mathbf{c}_1)$ will be maximized when $\mathbf{c}_1 = \mathbf{v}_1$ is the eigenvector associated to the largest eigenvalue λ_1 of the covariance matrix.

More specifically, the scaled eigenvector is known as the “first principal component” of the dataset and it explains a proportion of the variance equal to

$$EVR_1 = \frac{\lambda_1}{\sum_{n=1}^N \lambda_n}$$

Example: Example of principal component analysis in $\mathbb{R}^2$

Consider the following dataset:

$$\mathbf{x}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad \mathbf{x}_2 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \mathbf{x}_3 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Start by computing the sample mean $\bar{\mathbf{x}}$ to obtain the mean-centered data points $\mathbf{c}_p = \mathbf{x}_p - \bar{\mathbf{x}}$:

$$\bar{\mathbf{x}} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \Rightarrow \mathbf{c}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \mathbf{c}_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix} \quad \mathbf{c}_3 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$

This means that the empirical covariance matrix will be given by

$$\Sigma = \frac{1}{3} \sum_{p=1}^3 \mathbf{c}_p \mathbf{c}_p^T = \begin{pmatrix} \frac{2}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{2}{3} \end{pmatrix}$$

In particular, the eigenvalues of the covariance matrix can be found by solving

$$\begin{aligned} \det(\Sigma - \lambda \mathbf{I}) &= \det \begin{pmatrix} \frac{2}{3} - \lambda & \frac{1}{3} \\ \frac{1}{3} & \frac{2}{3} - \lambda \end{pmatrix} \\ &= \left(\frac{2}{3} - \lambda \right)^2 - \frac{1}{9} = \frac{4}{9} - \frac{4\lambda}{3} + \lambda^2 - \frac{1}{9} = \lambda^2 - \frac{4\lambda}{3} + \frac{1}{3} = 0 \end{aligned}$$

$$\Delta = \frac{4}{9} > 0 \Rightarrow \lambda_{1,2} = \frac{\frac{4}{3} \pm \frac{2}{3}}{2} \Rightarrow \lambda_1 = 1, \quad \lambda_2 = \frac{1}{3}$$

Using these eigenvalues, solving $(\Sigma - \lambda \mathbf{I})\mathbf{X} = 0$ allows to find the following eigenvectors:

$$\begin{aligned} \mathbf{v}_1 &= \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \text{ which corresponds to principal component } \mathbf{v}_1 = \begin{pmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{pmatrix} \\ \mathbf{v}_2 &= \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \text{ which corresponds to principal component } \mathbf{v}_2 = \begin{pmatrix} \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} \end{pmatrix} \end{aligned}$$

In particular, the proportion of data variance explained by the two components will be

$$EVR_1 = \frac{\lambda_1}{\lambda_1 + \lambda_2} = 0.75 \text{ and } EVR_2 = \frac{\lambda_2}{\lambda_1 + \lambda_2} = 0.25$$

It should be noticed that the same reasoning can be applied to the other basis vectors when analysing the other principal components.

4.3.1 Eigenvalue decompositions

For a real-valued and symmetric covariance matrix Σ , one can compute an orthonormal matrix $\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_d]$, which is obtained by stacking the eigenvectors of the covariance matrix, and a diagonal matrix Λ , which contains the eigenvalues of the covariance matrix, to formulate

$$\Sigma = \mathbf{V}\Lambda\mathbf{V}^T, \text{ implying that } \Sigma \text{ is always positive semi-definite.}$$

4.4 Applications of principal component analysis

Principal component analysis can be used as an unsupervised learning algorithm that allows to compress high-dimensional data onto a lower-dimensional space, such as a linear manifold, by using the $k \ll d$ most relevant components to create uncorrelated features that can be exploited to simplify learning tasks or discover data patterns.

Part II

Network Science

Chapter 5

Understanding networks

5.1 The purpose of network science

A network is a complex system whose components interact according to three main properties:

1. **Adaptation:** The components of the system mutate and self-organize in order to maintain certain properties.
2. **Non-linearity:** A change in the input is not proportional to a change in the output.
3. **Emergence:** The system as a whole shows some properties that the individual components do not possess.

Example: Schelling's segregation model

In Schelling's segregation model, agents wish that a fraction B_a of their neighbourhood, which consists of the neighbouring agents, is from the same group.

At each round, every agent checks the fraction of matching agents B and, if $B < B_a$, it can relocate to a vacant spot, going on until every agent is satisfied.

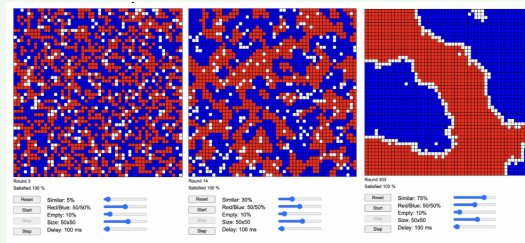


Figure 5.1: Example of Schelling's segregation model.

In particular, Schelling discovered a threshold B_{seg} such that choosing $B_a < B_{seg}$ leads to a random population configuration, whereas choosing $B_a \geq B_{seg}$ would result in segregation.

Overall, network science is useful for modelling complex systems by transforming high-dimensional data into feature representations that can be used for inference and pattern discovery.

5.2 A formal definition of networks

A network can be formally represented as a graph $G = (V, E)$ that is described using a set of nodes V and a set of edges E that connect pairs of vertices.

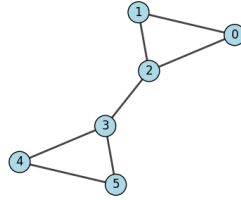


Figure 5.2: Example of a graph.

A graph is said to be “undirected” if its edges represent unordered pairs, which means that

$$E = \{\{u, v\} : u, v \in V, u \neq v\} \text{ and } \{u, v\} \in E \Leftrightarrow \{v, u\} \in E$$

By contrast, a graph is said to be “directed” if its edges indicate ordered pairs, which means that

$$E = \{(u, v) : u, v \in V, u \neq v\} \text{ and } (u, v) \in E \not\Leftrightarrow (v, u) \in E$$

5.2.1 Visualizing a network using an edge list

A network can be represented using an edge list in which each tuple represents a connection between two endpoints in the graph.

```
1 edges = [(0,1), (0,2), (1,2), (2,3), (3,4), (3,5), (4,5)]
2 g_py = ig.Graph(edges=edges, directed=False)
```

Listing 5.1: Creating a graph from an edge list, which requires $O(|V|)$ memory.

Alternatively, a network can be represented as a dataframe in which each row corresponds to an edge and identifies its source and target nodes.

```
1 edge_list_py = pd.DataFrame({
2     "source": [0, 0, 1, 2, 3, 3, 4],
3     "target": [1, 2, 2, 3, 4, 5, 5]
4 })
5 g_py = ig.Graph.DataFrame(edge_list_py, directed=False)
```

Listing 5.2: Creating a graph from a dataframe.

5.2.2 Visualizing a network using an adjacency matrix

It is possible to represent the connections in a network using an adjacency matrix in which $A_{i,j} = 1$ if and only if $(i,j) \in E$ and $A_{ii} = 0 \forall i$.

```
1  adj_matrix = [  
2      [0, 1, 1, 0, 0, 0],  
3      [1, 0, 1, 0, 0, 0],  
4      [1, 1, 0, 1, 0, 0],  
5      [0, 0, 1, 0, 1, 1],  
6      [0, 0, 0, 1, 0, 1],  
7      [0, 0, 0, 1, 1, 0]  
8  ]  
9  g_from_matrix = ig.Graph.Adjacency(adj_matrix, mode=ig.ADJ_UNDIRECTED)
```

Listing 5.3: Creating a graph from an adjacency matrix, which requires $O(|V|^2)$ memory.

Additionally, by considering at the definition of an adjacency matrix, it is possible to state that a graph $G = (V, E)$ is undirected if and only if its adjacency matrix is symmetric, meaning that $A^T = A$.

5.2.3 Visualizing a network using an adjacency list

It is possible to summarize the connections in a network using an adjacency list that, for each vertex in the graph, stores a list of its direct neighbours.

```
1  adj_list = [  
2      [1, 2],  
3      [0, 2],  
4      [0, 1, 3],  
5      [2, 4, 5],  
6      [3, 5],  
7      [3, 4]  
8  ]
```

Listing 5.4: Representing a graph using an adjacency list, which requires $O(|V| + |E|)$ memory.

It should be noticed that adjacency lists are not compatible with the **igraph** library, meaning that a conversion into an edge list or an adjacency matrix is required in order to plot a graph.